

Les pools de thread (épisode II)



Fabrice.Kordon@lip6.fr



En guise d'introduction...

2

ThreadFactory

- Interface
- L'une des interfaces de gestion de threads
- Alternative à l'instanciation de la classe Thread

Par défaut dans la classe Executors

- `static ThreadFactory defaultThreadFactory()`
 ▶ A priori utilisée par défaut
 ▶ `static ThreadFactory privilegedThreadFactory()`
- `static ExecutorService newCachedThreadPool(ThreadFactory tf)`
- `static ExecutorService newFixedThreadPool(int nThreads, ThreadFactory tf)`
- `static ExecutorService newSingleThreadExecutor(ThreadFactory tf)`



Un exemple simple

3

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

class UneThread implements Runnable {
    private String message;
    private int monNumero;

    UneThread(int id, String s){
        this.message=s;
        monNumero = id;
    }

    public void run() {
        this.trace("je dis " + message);
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        this.trace("bye bye");
    }

    private void trace (String m) {
        System.out.println ("UneThread " + monNumero + " : " + m);
    }
}
```

Un exemple simple

3

```
public class ExempleExecutor3 {
    static int nbThreads;

    public static void main(String[] args) {
        try {
            nbThreads = Integer.parseInt(args[0]);
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.err.println ("Arguments requis, nombre de threads.");
            return;
        }

        // Attention, ce "pool par défaut" est virtuellement infini
        for (int i = 0; i < nbThreads; i++) {
            Executors.defaultThreadFactory().newThread(
                new UneThread(i, "bonjour de " + i)).start();
        }
        System.out.println("Je n'ai plus rien à faire...");
    }
}
```

Quelques exécutions...

4

```
$ java ExempleExecutor3 7
UnThread 0 : je dis bonjour de 0
UnThread 2 : je dis bonjour de 2
UnThread 1 : je dis bonjour de 1
UnThread 4 : je dis bonjour de 4
UnThread 3 : je dis bonjour de 3
Je n'ai plus rien à faire...
UnThread 5 : je dis bonjour de 5
UnThread 6 : je dis bonjour de 6
UnThread 6 : bye bye
UnThread 5 : bye bye
UnThread 4 : bye bye
UnThread 0 : bye bye
UnThread 1 : bye bye
UnThread 2 : bye bye
UnThread 3 : bye bye
```

Quelques exécutions...



Mais bon...



Ne l'a-t-on pas cherché?

```
$ java ExempleExecutor3 2500
UnThread 0 : je dis bonjour de 0
UnThread 3 : je dis bonjour de 3
UnThread 2 : je dis bonjour de 2
UnThread 1 : je dis bonjour de 1
UnThread 5 : je dis bonjour de 5
UnThread 4 : je dis bonjour de 4
UnThread 6 : je dis bonjour de 6
UnThread 7 : je dis bonjour de 7
```

...

```
UnThread 2022 : je dis bonjour de 2022
UnThread 2023 : je dis bonjour de 2023
```

```
Exception in thread "main" java.lang.OutOfMemoryError: unable to create new native thread
    at java.lang.Thread.start0(Native Method)
    at java.lang.Thread.start(Thread.java:714)
    at ExempleExecutor3.main(ExempleExecutor3.java:47)
```

```
UnThread 2 : bye bye
UnThread 7 : bye bye
```

...

```
UnThread 2022 : bye bye
```

```
ExempleExecutor3 7
0 : je dis bonjour de 0
2 : je dis bonjour de 2
1 : je dis bonjour de 1
4 : je dis bonjour de 4
UnThread 3 : je dis bonjour de 3
Je n'ai plus rien à faire...
UnThread 5 : je dis bonjour de 5
UnThread 6 : je dis bonjour de 6
UnThread 6 : bye bye
UnThread 5 : bye bye
UnThread 4 : bye bye
UnThread 0 : bye bye
UnThread 1 : bye bye
UnThread 2 : bye bye
UnThread 3 : bye bye
```

Les groupes de threads

5

La classe ThreadGroup

- Regroupement d'un ensemble de threads
- Possibilité d'inclure des ThreadGroup dans un ThreadGroup
 - ▶ Hiérarchie arborescente
 - ▶ Les threads peuvent accéder aux caractéristiques du groupe
 - ▶ Une thread dans un groupe ne peut pas «remonter» au groupe parent

RTFM!



Constructeurs

- ▶ ThreadGroup(String name)
- ▶ ThreadGroup(ThreadGroup parent, String name)

Manipulation

- | | |
|-------------------------------------|---------------------------|
| ▶ int activeCount() | ▶ String getName() |
| ▶ int activeGroupCount() | ▶ ThreadGroup getParent() |
| ▶ int enumerate(Thread[] list) | ▶ void interrupt() |
| ▶ int enumerate(ThreadGroup[] list) | ▶ void list() |
| ▶ int getMaxPriority() | ▶ String toString() |
| ▶ void setMaxPriority(int prio) | |

Manipulation avancée des threads

6

De nouvelles primitives

Constructeurs

- ▶ Thread(Runnable target, String name)
- ▶ Thread(ThreadGroup group, Runnable target)
- ▶ Thread(ThreadGroup group, Runnable target, String name)



Manipulation

- ▶ static Thread currentThread()
- ▶ String getName()
- ▶ void setName(String name)
- ▶ int getPriority()
- ▶ void setPriority(int newPriority)



Sur les priorités

Très dépendant de l'implémentation
Utilisation de constantes
Thread.MIN_PRIORITY
Thread.NORM_PRIORITY
Thread.MAX_PRIORITY

Utile pour debug

- ▶ static void dumpStack()

En guise de conclusion...

7

Mécanismes assez souples

- Permet de structurer hiérarchiquement les threads
 - ▶ Et les sous-classes de Thread

-  Suppression du groupe
 - ▶ Via `interrupt()`

Gestion de priorités

- Déterminer la priorité «maximum» du groupe